

Reflection Journal Report for Midterm

Prepared for Prof. Vishwantha Rao, HCC ITAI 1371 Machine Learning

Prepared by FA26_ML_6171_16347 3_ITAI1371 Group

03-Oct-2026

The purpose of this report is for each student to reflect on their learning journey for ITAI 1371 Machine Learning (ML) MidTerm. This Reflection Journal Report consists of separate parts for each student in the group. Each student in the group was responsible for their own content and report formatting in each of the sections below.

This reflection journal entry is for the Midterm Exam was prepared by Eric Roberson.

I completed this midterm in collaboration with my teammates who chose the Netflix dataset. I draft and rewrote the proposal based on feedback from Prof. Rao, and subsequently getting feedback from the Team. After rewriting the proposal, I built the Colab notebook, and prepared the cleaned datasets. One classmate, Irene “Alex” Ramos, reviewed my work with the IPYNB notebook as the team’s QAer.

Understanding Insights & New Breakthroughs

My biggest breakthrough was seeing that the order of preprocessing matters as much as the techniques. I honestly thought cleaning was one step: fill blanks, scale, encode. Working alone, I had to work out why computing a median or scaling range on the full dataset leaks test information into training. So I split 70/30 first and fit every imputer, transformer, encoder, and balancing step on the training data only. I also learned the difference between *scaling* (changing the range, using min-max) and *normalization* (changing the shape of a skewed distribution, using a Yeo-Johnson transform).

Analyzing the Problem / Problem Solving Strategies

I started by deciding what question the data could answer: can we predict whether a Netflix movie is appropriate for a children's profile? I took on the persona of a Netflix employee working on the Governance, Risk, & Compliance Team. My goal was to both protect kids from viewing prohibited content and also to protect the company from lawsuits and fines, which was articulated in the proposal. No column says "kid-safe," so I built the target from the rating field. I coded G, PG, and the TV-Y and TV-G/PG ratings as kid-safe, and PG-13, R, NC-17, TV-14, and TV-MA as mature. This was not captured in the proposal because I could not determine this until actually working with the Colab notebook. Rather than guessing, movies were excluded if rated NR or UR, had no rating, or had no reliable label. I then checked the raw data for hidden problems: missing info in director, cast, and country, and a few records with a run time typed into the rating column. Every fix was done in Python so the original Kaggle file was never changed by hand. I measured class balance before deciding whether to correct it.

Real-World Applications

A model like this could help a streaming service like Netflix suggest titles for a kids' profile, or flag unrated titles for human review. It might also be used to audit the work performed by humans in the real-world. The costs of mistakes are not equal: labeling a mature movie as kid-safe is worse than missing a kid-friendly one, so the final evaluation should emphasize precision on the kid-safe class, not just accuracy. In my work in technical risk and controls, the same habits apply: documenting every data decision, keeping results reproducible, and avoiding a control that grades its own homework, which is what target leakage is. One of the most important aspects of my job is to defend business partners against audit and to provide evidence of compliance with technical control standards and control procedures. I can see how ML could be used to augment work if I had access to the right blob of data.

Connecting New & Prior Knowledge

Labs 04 and 05 covered missing values, scaling, encoding, and class balance as separate topics. This project required combining them in the right order and justifying each choice. I tied *target leakage* to dropping the rating column from the model inputs, since it was used to build the label. I also applied the idea that accuracy can mislead on imbalanced data, which is why I balanced only the training set and left the test set with its natural class mix.

Emerging Questions

- Rating systems vary by country and between movies and TV. How reliable is a rating as ground truth for "kid-safe"?
- Genre tags like "Children & Family Movies" are very informative. Will models lean on them so heavily that other features barely matter? What is the risk of models doing so?
- I used random oversampling because the data is mostly one-hot and multi-hot encoded. Would SMOTE or class weights do better in the finals?
- Does excluding unrated movies bias what the model learns?
- Could the description text add predictive power through TF-IDF features?

Changing Approach & Critical Thinking

My approach changed as I built the project. I initially narrowed it from all titles to movies only and defined a kid-safe target. My proposal first called for seven simple features and EDA only on the training set. While building the notebook, I expanded the features (years between release and arrival on Netflix, missing-value flags, a duration bucket) and refined the split to be sure raw data is inspected first while every learned value comes only from the training set. I widened the kid-safe definition beyond G and PG to include the TV equivalents, because leaving them out would have mislabeled clearly child-oriented titles. I made minor revisions to the proposal to match the final notebook. Throughout, I questioned each step: whether a value was an error or a real outlier, whether a transformation was justified, and whether a feature could leak the answer. Alex's QA did not return anything to be changed with the IPYNB file.

This reflection journal entry is for the MidTerm Exam and was prepared by Irene(Alex) Ramos:

Understanding Insights & New Breakthroughs

While developing our midterm proposal, I gained a clearer understanding of how exploratory data analysis prepares a dataset for machine learning. Before this project, I mostly thought of data cleaning as removing empty information or correcting obvious errors. I now understand that preprocessing decisions can directly affect the accuracy and fairness of a future model. One of my biggest breakthroughs was learning that the original rating column must be removed from the model inputs after it is used to create the “kid_safe” target. Keeping it would cause target leakage because the model would essentially be given the answer it is supposed to predict.

I also learned why the training and testing data should be separated before completing EDA. We plan to use 70% of the data for training and reserve 30% for testing. Analyzing the training set while leaving the test set untouched will give us a more honest evaluation of the model later.

Analyzing the Problem / Problem-Solving Strategies

Our project asks whether the characteristics of a Netflix movie can help classify it as potentially appropriate for a children’s profile. To make the problem manageable, we decided to filter the original Netflix dataset so that it contains only movies rather than both movies and television shows. We will create a binary target called “kid_safe”, with G- and PG-rated movies coded as 1 and clearly mature ratings coded as 0. Movies with missing or unclear ratings will be separated for review instead of being automatically labeled.

Our problem-solving strategy is to complete each step through reproducible Python code. We will inspect the dataset for duplicates, missing values, incorrect data types, skewed numerical distributions, and class imbalance. Instead of applying the same solution to every missing value, we will choose an appropriate method for each feature. For example, some missing categorical values may be assigned an “Unknown” category, while records missing information needed to create the target may need to be excluded from the modeling dataset.

Real-World Applications

This project has practical applications for streaming platforms, parents, and families. A classification model could help identify movies that may be suitable for a children’s profile, especially when rating information is incomplete, inconsistent, or unavailable. Features

such as genre, description, duration, release year, country, director, and cast may contain patterns that help support this decision.

However, I also recognize that a machine learning prediction should not replace parental judgment or an official content rating. A movie may have a G or PG rating and still contain themes that some families consider inappropriate. Therefore, the model would work best as a recommendation or screening tool rather than as the final authority on whether a child should watch a movie.

Connecting New & Prior Knowledge

This proposal helped me connect concepts from earlier coursework to a complete machine learning workflow. I had previously worked with pandas, missing values, categorical data, and visualizations, but this project helped me understand how those skills work together. Loading the original dataset, inspecting its structure, filtering records, engineering new features, encoding categories, and visualizing distributions are not separate activities. Each step contributes to creating reliable data for a future model.

I also connected what I know about classification to the importance of selecting a clear target variable. Changing our original idea from predicting several content-rating categories to a binary “kid_safe” classification made the problem more focused. It also helped us define which rows belong in the project and which features might be useful predictors.

Emerging Questions

As we continue the project, several questions remain. How should ratings such as TV-PG or TV-Y7 be handled if they appear on records labeled as movies? Should PG-13 movies automatically be considered unsafe, or should they form a separate category for human review? How many G- and PG-rated movies will remain after filtering, and will that create serious class imbalance?

I am also interested in finding out which engineered features will be the most useful. For example, will keywords from the description provide stronger information than genre or duration? How should columns containing multiple values, such as cast, country, and listed genres, be encoded without producing too many features? Another question is whether the definition of “kid-safe” is too subjective to be represented by one binary target.

Changing Approach & Critical Thinking

Our approach became more specific as we reviewed the assignment requirements and thought more carefully about the data. Instead of manually editing the original dataset, we decided that every transformation should be completed through Python so that the process can be repeated and verified. We also changed our approach by separating movies from television shows, defining a clear binary target, and planning to reserve an untouched test set.

The most important change in my thinking was realizing that preprocessing methods should be chosen only after examining the data. We should not automatically scale every numerical variable, remove every row with a missing value, or balance the classes without evidence that those actions are necessary. We will first study the training data and then document the reasoning behind each decision. This project has encouraged me to think more critically about how data choices influence the final model and to focus on building a process that is transparent, reproducible, and appropriate for the problem.

This reflection journal entry is for the Midterm Exam and was prepared by Yuanyuan Kou:

In this project, I reviewed our proposal and the instructor's requirements. Our goal is to prepare Netflix movie data for a future model that predicts whether a movie may be suitable for children. I learned that this midterm focuses on understanding and cleaning the data before building models.

The previous labs helped me understand missing values, encoding, scaling, and feature engineering. However, applying these ideas to our own project was more difficult than following a lab example. I needed more time to read the notebook and understand why each step was needed.

One important thing I learned is that we must split the data into 70% training data and 30% testing data before doing EDA. We should use only the training data to study distributions and choose how to fill missing values. For this assignment, we should keep the testing data untouched.

I also learned about target leakage. We use the rating column to create the kid_safe label, so we must remove rating from the model inputs. Missing or unrated movies should not automatically be labeled unsafe because we do not know their correct labels.

I am still learning how the full process works, especially class balancing and encoding. Reviewing this project helped me connect the ideas from our labs to a real dataset. My next step is to follow the updated notebook cell by cell and check the outputs carefully.

This reflection journal entry is for the Midterm Exam and was prepared by [CARMEN ESONO]:

Working on the Exploratory Data Analysis (EDA) proposal for our machine learning project provided valuable insight into the critical role that data preprocessing plays in the machine learning workflow. Our group's objective is to prepare the Netflix dataset to classify whether a movie is appropriate for a children's profile based on key characteristics such as genre, duration, release year, country of origin, and description length.

A key takeaway from this project phase was the concept of preventing data leakage. Constructing our binary target directly from the existing `rating` column required strictly removing `rating` from the predictor features so the future model relies on true content attributes rather than explicit maturity codes. Furthermore, defining a strict 70/30 stratified train-test split *before* fitting any imputation, power transformations (such as Yeo-Johnson), scaling, or class balancing ensured that learning happens strictly on training data without compromising the untouched evaluation set.

Through this proposal process, I learned how raw metadata contains hidden nuances like missing values in directors, cast, or country fields that must be handled systematically rather than being discarded or ignored. Establishing a reproducible, code-driven pipeline in Python reinforces how important rigorous data cleaning and feature engineering are to building reliable predictive models.

This reflection journal entry is for Midterm Exam and was prepared by [Xiomara Torres]:

Working on this midterm project helped me gaze into how much work goes into a machine learning project before a model is created. Throughout the semester we have learned about the process a machine learning model goes through. Working with the Netflix dataset helped me see how everything ties together when working with data.

I learned how important it is to prepare the data before using it to train a model. Our project focuses on determining whether a Netflix movie could be considered appropriate for a children's profile. The first step was to decide what information from the dataset would be useful to us and how we would define our outcome. Since the original rating is being used to determine whether a movie is considered kid-safe, we could not use the rating as an input for the model because it would give away the answer.

This project helped me understand the importance of separating the dataset into training and testing data. Before this project I understood the idea of a train-test split, but I did not fully understand why the order mattered so much. Keeping the testing data separate allows us to make decisions using the training data without allowing information from the test set to influence those decisions. This gave us an idea of how well the model performs data it has never seen before.

One question I still have is how much our definition of safety will affect the results. Movie ratings give us a starting point, but what parents consider appropriate for their children can vary drastically. I am also interested in seeing which features end up being the most useful for predicting the target and how much class imbalance could affect the model.

Overall, this midterm gave me an understanding of why exploratory data analysis and preprocessing are such important parts of machine learning. My biggest takeaway is that building a model is not just about choosing an algorithm. The quality of the model depends heavily on how the data is understood, prepared, and evaluated before it is built in the first place.